AeroMorse Metro build guide: no breadboard, arcade button test switches

This guide builds an **AeroMorse**: a box that lets someone type and move the mouse on a computer using Morse code. They give a dot or a dash by **sipping or puffing** on a tube, or by pressing an **assistive-technology (AT) switch** plugged into the box. A small speaker beeps for each dot and dash. To the computer, the box is an ordinary USB keyboard and mouse.

This version has **no breadboard and no push buttons on the box**: the two switch sockets are wired straight to the Metro. Two arcade buttons become plug-in **test switches** instead, for a caregiver to check the box works before a real AT switch goes in. It's written for someone who has never built electronics before. Every step says what to do and what you should see. **Nothing needs soldering**: every part plugs in, pushes in or screws in.

Status: this build has been **assembled and tested** as a complete working AeroMorse, on 2026-11-10, with the parts in the list below (the two optional rows aside), running CircuitPython 10.3.1 and AeroMorse v1.33. The sensor, both switch jacks and the speaker all worked. It's based on the Metro section of the AeroMorse Build Guide (Appendix L) and on Adafruit's own documentation. The few things not yet tested are listed near the end.

How this guide is organized

1. **Before you start:** the parts, the tools, the words used, and the safety rules.
2. **Get the board ready** and install its software, before any wiring.
3. **Wire the parts** one at a time, then check the wiring against a checklist.
4. **Make two test switches** from arcade buttons and plug adapters.
5. **Test everything** with a short test program and the serial console.
6. **Install AeroMorse** and adjust how hard a sip or puff has to be.
7. **Optional: add a wireless display**, a second board with a screen that shows what the AeroMorse is doing.

Before you start

Parts

These are the parts of the tested build. The two marked optional weren't part of it: the wireless display and the enclosure screws.

Part	Qty	What it is	Where from	Price
Metro ESP32-S3 (#5500)	1	The "brain": a small computer board, the size of a playing card. Its sockets come already fitted.	Adafruit	$24.95

Part	Qty	What it is	Where from	Price
LPS33HW pressure sensor (#4414)	1	Measures sips and puffs through a tube.	Adafruit	$12.50
STEMMA QT cable, 50 mm (#4399)	1	A short cable with a small plug on each end. Connects the sensor to the Metro.	Adafruit	$0.95
Silicone tubing, 3 mm inside × 6 mm outside, 1 m	1	The tube for sipping and puffing. Choose the **3x6mm (3.3Ft)** size. Sold under the uxcell name.	Amazon	$7.59
Push-to-connect tube joiner, 6 mm, 15-pack	1 pack (use 1)	A small plastic connector that joins two pieces of 6 mm-outside tubing: push a tube in to connect it, press the blue ring to release it. Choose size **PU-6mm**. Sold under the ATMLCOER name.	Amazon	$7.99
STEMMA Speaker (#3885)	1	A small speaker with its own amplifier and a volume screw.	Adafruit	$5.95
Speaker cable (#3893)	1	3-wire cable: a plug for the speaker on one end, three pins on the other. Sold separately from the speaker.	Adafruit	$1.25
3.5 mm jack boards with bent pins, 5-pack	1 pack (use 2)	Small boards with a headphone-style socket, where an AT switch plugs in. Choose the **With bent needle** option. Each has a row of 5 pins, bent to stick out sideways and numbered 1 to 5: pin 1 = sleeve (ground), pin 5 = tip. Sold under the Teansic name; the board is printed SmartCool.	Amazon	$8.99
Female/Male "extension" jumper wires	4	Wires with a socket on one end and a pin on the other, from the jack boards straight to the Metro. 20 cm (8 in) is long enough.	You have these	—

Part	Qty	What it is	Where from	Price
Arcade buttons, 30 mm, 7-pack	1 pack (use 2)	Large round push buttons with a click switch underneath, plus a 2-wire cable for each. Two become the plug-in test switches in Part 7. Sold under the Fosiya name, in red or black.	Amazon	$14.99
3.5 mm mono plug adapters, 5-pack	1 pack (use 2)	A headphone-style plug with two screw terminals, marked **+** and **−**, so wires attach with a screwdriver instead of solder. Choose the **1/8" TS Mono Male** option. Sold under the AKVIBG name. The stereo version, with three screws, works too (tested); see Part 7.	Amazon	$5.39
USB-C **data** cable	1	Connects the Metro to the computer. A charge-only cable will **not** work.	Check you have one	—
Two AT switches with 3.5 mm plugs	2	The switches the user presses. Each ends in a headphone-style plug.	You have these	—
ESP32-S3 Reverse TFT Feather (#5691)	1 (optional)	A second small board with a colour screen built in. Only for the **wireless display** in Part 13: it shows what the AeroMorse is doing, with no wires between them. It needs its own power: a USB charger, a power bank or a computer.	Adafruit	$24.95
Fasvicna self-tapping screw kit, 750 pcs	1 kit (optional; use the M2 sizes)	Small black Phillips (cross-head) pan-head screws that cut their own thread into plastic, in a clear box with compartments. Choose the **M1.7M2M2.3M2.6M3 750pcs** option. Only for the enclosure (case) being designed for this build, which uses M2 screws.	Amazon	$7.99 (list $9.99)

Prices were read on 2026-10-27 (Adafruit) and between 2026-11-04 and 2026-11-10 (Amazon) and change often. The Adafruit wishlist "AeroMorse Metro" also holds parts this build doesn't use: the mounting plate (#275), the 2.5 mm tubing (#3659), the push-button pack (#4431), the half-size

breadboard (#64), the TRRS terminal block (#2915), the piezo (#1740) and the 3.5 mm plug terminal block (#2790).

About the enclosure screws. The enclosure is still being designed, so this guide doesn't cover putting it together yet. The kit has 15 sizes that look almost the same, each in its own compartment. The enclosure uses **M2**: a 2 mm thick screw. The kit has M2 in three lengths, 4 mm, 5 mm and 6 mm, with 50 of each. Put each screw back in its own compartment so the sizes don't get mixed up. "Self-tapping" means the screw cuts its own thread as it goes into plastic. Turn it slowly and stop as soon as it feels snug: turning further strips the plastic, and the screw won't hold.

Tools

- **A computer** (Windows, Mac or Linux) with a free USB port and internet access.
- **A multimeter** with a continuity setting (the one that beeps when its two probes touch). Used once, to find the right pins on the jack boards.
- **A small screwdriver**, to tighten the plug adapters' terminal screws and turn the speaker's volume screw. For the enclosure screws you need a **small Phillips (cross-head) screwdriver, size #0** (also written PH0), the standard size for M2 pan-head screws. A tip that wobbles in the head will damage it.
- **Flush cutters or sharp scissors**, to cut the tubing square and cut connectors off the arcade button cables.
- **Wire strippers**, to strip the ends of the test switches' wires (Part 7).

Words used in this guide

Word	Meaning
Socket	A small hole in the black strips along the Metro's edges. A wire's pin pushes into it. Each socket has a label printed beside it on the board.
Pin	The short metal spike on the end of a wire or on a jack board.
Jumper wire	A short wire for connecting parts. **Male-to-male** has a pin on both ends; **male-to-female** has a socket on one end that pushes onto a pin.
GND (ground)	The "return" connection every part needs. On the Metro, sockets labeled **GND**.
3.3	The Metro's 3.3-volt power socket, labeled **3.3**.
VHI and VIN	The Metro's higher-voltage sockets. **Nothing in this build goes into them.**
D5, D6, A0	Sockets the AeroMorse program uses: D5 for dot, D6 for dash, A0 for the speaker. On the board they're labeled **5**, **6** and **A0**.
Screw terminal	A small green block with a screw on top. Push a wire into its hole and tighten the screw to clamp it.
Tip and sleeve	Parts of a headphone-style plug. The **tip** is the very end; the **sleeve** is the long metal part nearest the plastic handle.

Word	Meaning
CIRCUITPY	The drive that appears on your computer when the Metro is plugged in, like a USB stick. You copy files onto it.
Serial console	A text window on your computer that shows messages from the Metro.
hPa	Hectopascal, the unit of air pressure the sensor reports.

Safety rules

- **Unplug the USB cable before you add, move or remove any wire.**
- **Never put a wire in VHI or VIN.** They carry 5 volts, more than the Metro's signal sockets can take.
- Push pins in straight and gently. If one won't go in, check you have the right socket rather than forcing it.
- Hold boards by their edges.

Part 1: Get the Metro ready

1. Take the Metro out of its bag and set it on a clean, dry table that isn't metal. Metal under the board could connect its pins together.
2. Until it goes into an enclosure, keep it there while you build: the other parts connect to it with cables and wires, and nothing needs to be screwed down.

Part 2: Install CircuitPython

CircuitPython is the system that runs AeroMorse on the Metro. Install it now, before any wiring, so you know the board works.

1. Find the small **On/Off switch** next to the round power socket, labeled **ON** and **OFF** (see the photo in Part 4), and slide it to **ON**. Adafruit says that at OFF the board's 3.3 V power and its main chip are switched off, so nothing works.
2. Plug the Metro into the computer with the USB-C **data** cable. The small green light labeled **ON** should light up.
3. Find the **Reset** button: the small button in the board's top-left corner, labeled **Reset**. Press it twice quickly, like a double-click.
4. A new drive appears on the computer whose name ends in **BOOT**. If it doesn't, try the double-press again a little faster or slower.
5. In a web browser, go to circuitpython.org/board/adafruit_metro_esp32s3 and download the **.uf2** file for the latest stable version (10.3.1 when this guide was written).
6. Drag the .uf2 file onto the BOOT drive. The drive disappears, the board restarts, and after a few seconds a drive called **CIRCUITPY** appears. That means it worked.

7. If no BOOT drive ever appears, the Metro is missing a small start-up program. The same circuitpython.org page explains how to install it, using the Metro's **Boot** button (beside the word Metro on the board).

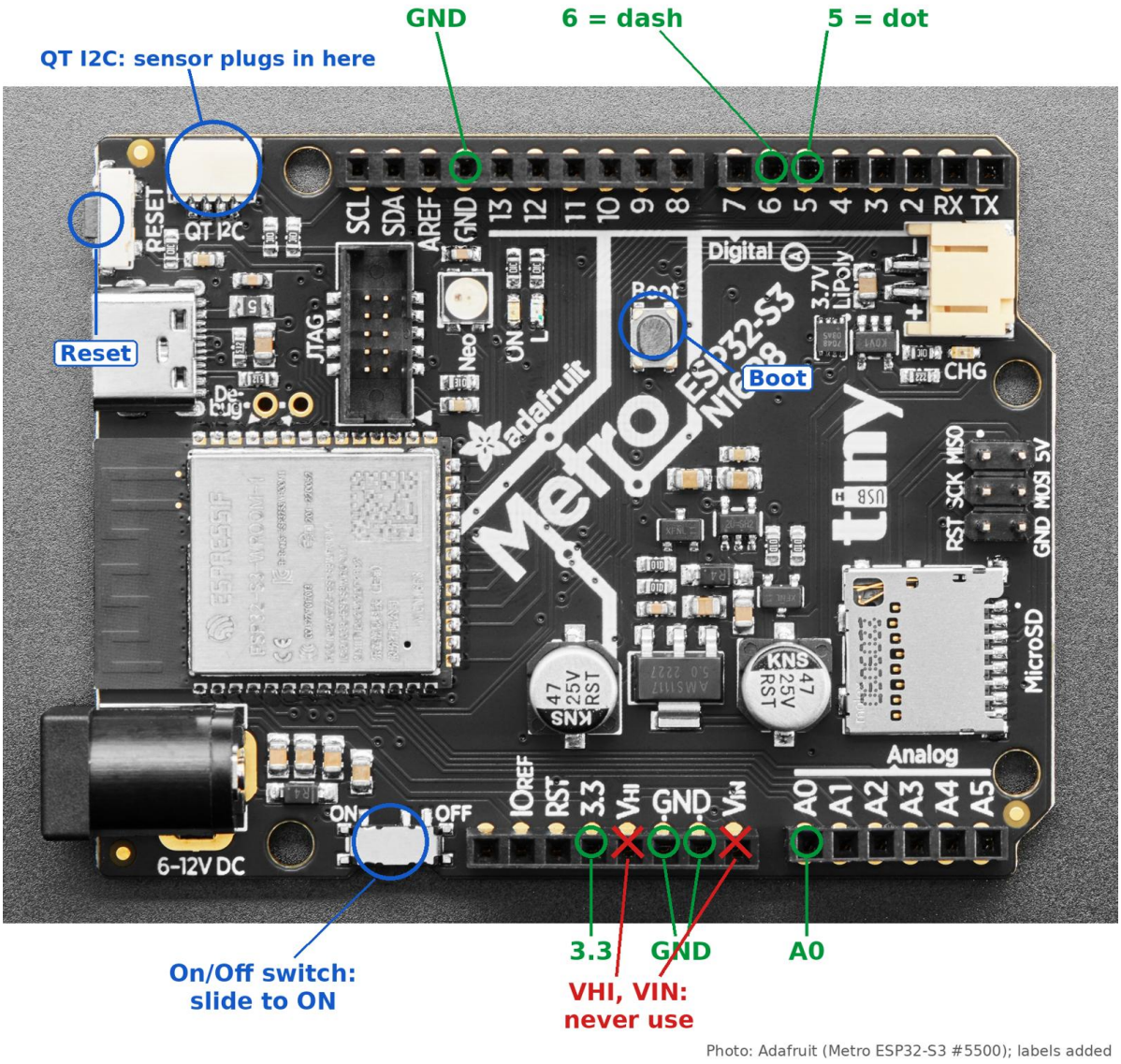
Part 3: Install the libraries

Libraries are small add-on files AeroMorse needs. They come in one download called the library bundle.

1. Go to circuitpython.org/libraries and download the bundle for **10.x** (it matches CircuitPython 10.3.1).
2. Open the downloaded .zip file. Inside is a folder called **lib** with hundreds of files.
3. On the CIRCUITPY drive, open the folder called **lib**. If there isn't one, create a new folder and name it `lib`.
4. Copy these from the bundle's lib folder into CIRCUITPY's lib folder: the folders `adafruit_hid`, `adafruit_display_text`, `adafruit_bitmap_font`, `adafruit_bus_device` and `adafruit_register`, and the files `adafruit_lps35hw.mpy` and `neopixel.mpy`.
5. Two of these are easy to miss. `adafruit_lps35hw.mpy` is the sensor's file even though the sensor is called LPS33HW; one file covers both. `neopixel.mpy` has no adafruit_ at the start, so it's near the bottom of the list.

Part 4: Find the sockets you will use

Unplug the USB cable. Every socket has its label printed beside it. Compare your board with the photo:



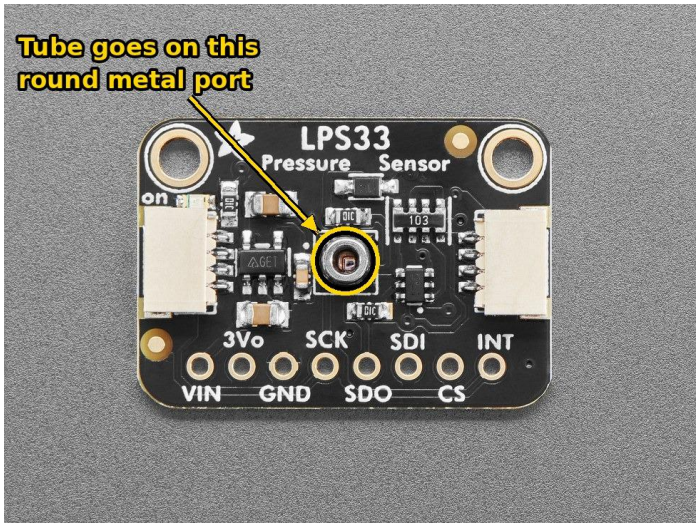
The Metro ESP32-S3. Green: sockets this build uses. Red: never use. Blue: switch, buttons and the sensor socket. Photo: Adafruit; labels added.

- **Top row:** SCL, SDA, AREF, **GND**, 13, 12, 11, 10, 9, 8, then 7, **6**, **5**, 4, 3, 2, RX, TX. You use **5** (dot), **6** (dash) and this **GND**.
- **Bottom row, left part:** first one socket with **no label** (at the end nearest the round power socket), then IOREF, RST, **3.3**, VHI, **GND**, **GND**, VIN. You use **3.3** and **both GND** sockets. The second GND is right next to VIN: count carefully. **Never VHI or VIN.** When counting, start from IOREF, not from the unlabeled socket.
- **Bottom row, right part: A0** to A5, labeled Analog. You use **A0** (speaker).
- **QT I2C:** the small white socket in the top-left corner beside the Reset button, where the sensor plugs in.
- **On/Off switch:** bottom left, beside the round power socket.

If a label is hard to read, use your phone's camera zoom or a magnifier.

Part 5: The pressure sensor and tube

Plug in the sensor



The LPS33HW sensor. The tube goes on the round metal port in the middle (circled). The white sockets on each side are its two STEMMA QT sockets. Photo: Adafruit; label added.

1. Push one end of the STEMMA QT cable into either of the sensor's two small white sockets; both work the same. The plug only fits one way round; don't force it.
2. Push the other end into the Metro's small white socket labeled **QT I2C**, in the top-left corner beside the Reset button.

Attach the tube

The tube goes straight onto the sensor's metal port, as in Adafruit's sip-and-puff guide. Adafruit calls this "definitely an off label use of the sensor port", but it's their tested method and it's what the tested build uses.

Tube size. The tested build uses silicone tubing **3 mm inside diameter (I.D.) and 6 mm outside (O.D.)**. It pushes snugly over the port. The closest inch size is **1/8 in I.D.** (3.2 mm). **Don't use aquarium air tubing**: it's usually 3/16 in I.D. (4.8 mm), far too loose.

The joiner. A short piece of tube stays on the sensor, and the user's tube plugs into it through the push-to-connect joiner. That way the user's tube can be taken off for cleaning or swapped for a new one without touching the sensor. The joiner is made for 6 mm O.D. tubing, the same as this tube's outside.

The push-to-connect joiner: push a tube into either end; press the blue ring to release it. Its photo is on the Amazon listing (see Sources).

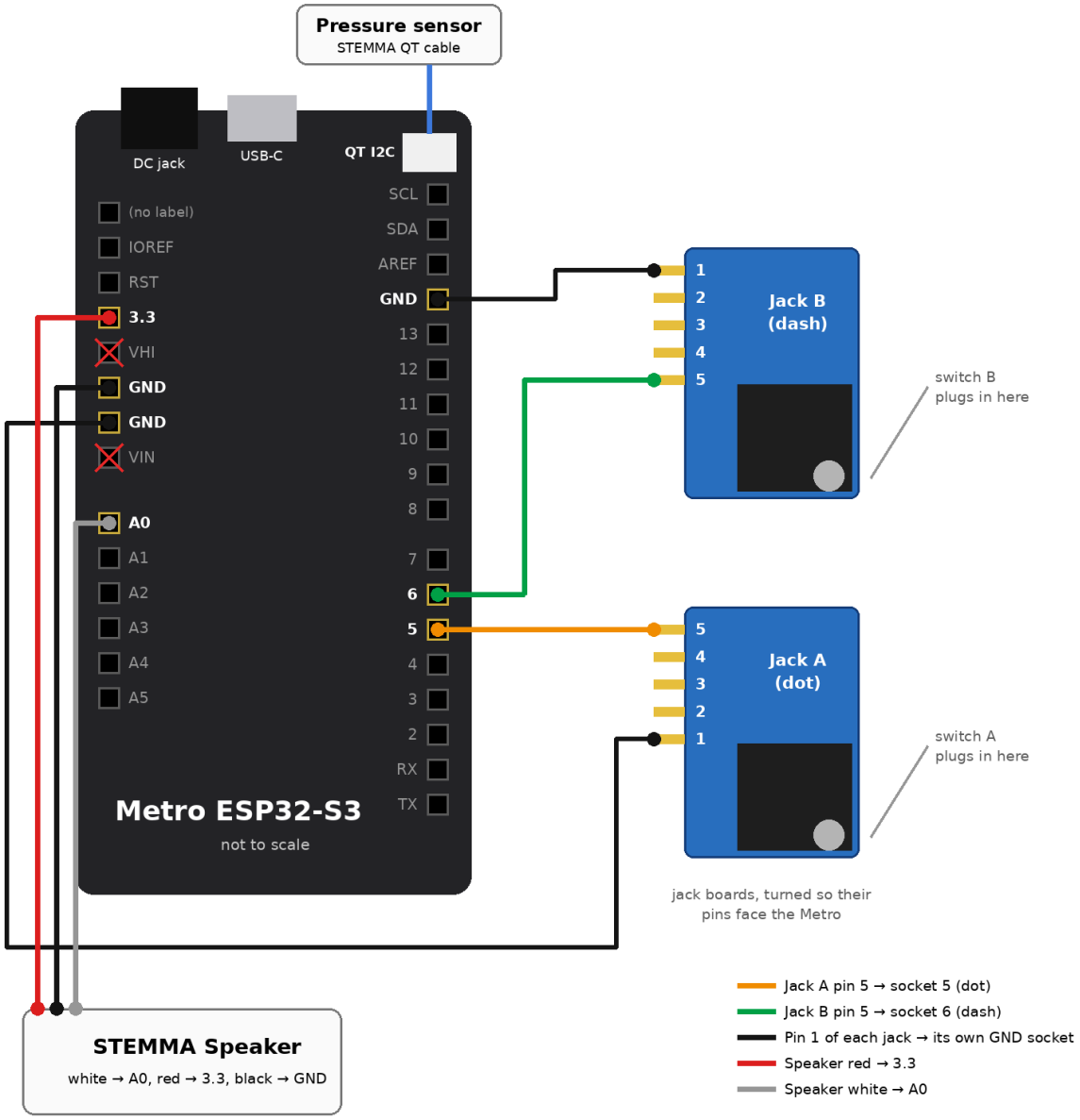
1. Cut a short piece of tubing, about 5 cm (2 in) long. Cut both ends flat and square, each in one stroke.
2. Push one end of the short piece onto the sensor's round metal port, as far down toward the board as it goes, twisting gently as you push. Tug gently: it should stay on.

3. Push the other end of the short piece firmly into one end of the joiner until it stops. Tug gently: it should hold.
4. Cut the user's tube long enough to reach from where the box will sit to the user's mouth, with both ends square. Push one end into the other end of the joiner until it stops.
5. **To take the user's tube off:** press the blue ring on that end of the joiner in toward the joiner and hold it, then pull the tube out.

A tube that goes in someone's mouth needs regular cleaning or replacing. With the joiner, the user's tube can be swapped in seconds.

Part 6: The AT switch jacks

This drawing shows the finished wiring for Parts 6 and 8. Use it as a map while you follow the steps.



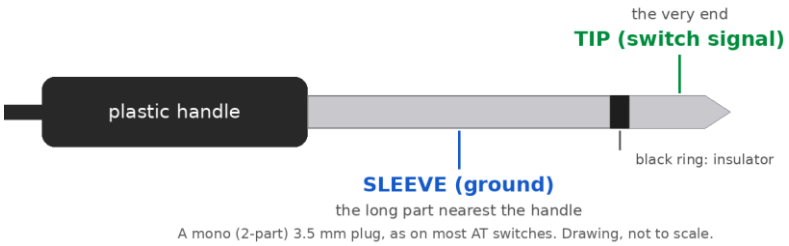
No breadboard, no buttons. Every jack wire is male-to-female: socket end on the jack pin, pin end in the Metro. Pins 2, 3 and 4 stay empty.

The complete wiring, with no breadboard. Each jack board connects straight to the Metro with two male-to-female wires. Orange: jack A pin 5 to socket 5 (dot). Green: jack B pin 5 to socket 6 (dash). Black: grounds, each to its own GND socket. Red and grey: speaker power and signal.

The jack boards connect to the Metro with **male-to-female wires**: the female end pushes onto a jack-board pin, the male end into a Metro socket. Each jack board needs two wires, so the build uses four.

The jack board with bent pins: the pins are numbered 1 to 5 in small print, and the AT switch plugs into the socket. Its photo is on the Amazon listing (see Sources).

Check which pins are tip and sleeve



The two metal parts of an AT switch's plug.

Jack pin	What it is	Used here?
1	Ground: the sleeve (the long barrel)	Yes: to a GND socket
2	Right channel: the ring	No
3	Plug-detection switch, terminal 1	No
4	Plug-detection switch, terminal 2	No
5	Left channel: the tip	Yes: to socket 5 or 6

This pinout is from the Amazon listing for these boards (B0FG24PBJJ, printed SmartCool 191768A_Y1912). An AT switch's mono plug has only a tip and a sleeve, so it uses pins 5 and 1. A quick check before wiring (if you have no AT switch yet, make the test switches in Part 7 first and use one of them):

1. Plug one AT switch into jack board A (for dot).
2. Set the multimeter to continuity (the setting that beeps). Touch its two probes together: it should beep.
3. Hold one probe on jack pin **5** and the other on pin **1**. Press the switch: the meter should beep while it's pressed and stop when you let go.
4. If it doesn't beep, keep one probe on pin 1 and try pins 2, 3 and 4 in turn, pressing the switch each time. Write down the pair that beeps and use those in place of 5 and 1 below.
5. Repeat with jack board B (for dash).

Wire the jacks

1. Make sure the USB cable is unplugged.
2. Push the **female** end of a male-to-female wire onto jack A's **pin 5**. Push its male end into Metro socket **5**. Jack A is now the **dot** socket.
3. Push another wire onto jack A's **pin 1**. Push its male end into the **second GND** socket in the Metro's bottom row: the one right next to **VIN**. Count carefully: **never VIN**. This wire runs around the end of the Metro; if it doesn't reach, use a longer wire.
4. Push a wire onto jack B's **pin 5**, and its male end into Metro socket **6**. Jack B is now the **dash** socket.

5. Push a wire onto jack B's **pin 1**, and its male end into the **GND** socket in the Metro's top row, between AREF and 13.
6. Lay both jack boards socket side up beside the Metro, pins facing it and sockets facing out, so the switches plug in from outside: jack B (dash) nearer the USB end and jack A (dot) below it, as in the drawing. Make sure no bare pin touches another pin or any metal.

Leave pins 2, 3 and 4 of each jack board empty.

Part 7: Make two test switches

A test switch is an arcade button with a headphone-style plug, so it plugs into a jack board just like a real AT switch. A caregiver can use the two test switches to check the box works, or to tell whether a problem is in the box or in the user's own switch. They also let you test the build before the AT switches arrive.

The arcade buttons: each has a click switch (the small box) clipped underneath, and a cable with a metal spade on each wire at one end and a small white connector at the other. Their photo is on the Amazon listing (see Sources).

The mono plug adapter: the plug, and two screw terminals marked + and –. Its photo is on the Amazon listing (see Sources).

Inside each arcade button is a **click switch**: pressing the button closes it, and letting go opens it again, exactly like an AT switch. Its metal tabs are usually marked in tiny print: **C** or **COM** (common), **NO** (normally open) and sometimes **NC** (normally closed). This build uses **C** and **NO**. Leave NC empty.

1. Unpack two buttons and two of the cables. Use two buttons of different colors (the listing photo of the red pack also shows two white ones), or mark one with tape, so the caregiver can tell them apart.
2. The black ring on each button is a nut, used later only if you mount the button in a panel. Leave it where it is.
3. Look at the click switch on the bottom of the button for its markings. Push one of the cable's metal spades onto the tab marked **C** (or COM), and the other spade onto the tab marked **NO**. Push them all the way on; they're a tight fit. If there are only two tabs, use those two.
4. At the other end of the cable, cut off the small white connector with the flush cutters, as close to it as you can. It's made for game controller boards and doesn't fit the plug adapter.
5. With the wire strippers, strip about 5 mm (1/4 in) of plastic off the end of each of the two wires. Twist the bare strands of each wire together so no loose strand sticks out.
6. On a plug adapter, turn both terminal screws to the left (counter-clockwise) a few turns, until you can see the holes in the side of the green block open up.
7. Push one bare wire end into the hole under **+** and tighten that screw to the right until it's firm. Push the other wire into the hole under **–** and tighten it the same way. For a click switch it doesn't matter which wire goes where.
8. Tug each wire gently. If one slides out, loosen the screw, push the wire in further and retighten. Check that no bare strand touches the other terminal.
9. Make the second test switch the same way.

Stereo plug adapters work too (tested). If your plug adapters have three screws, marked ⏚ (ground), **R** and **L**, they are stereo plugs. Put one wire under **L** (the tip) and the other under ⏚ (the sleeve), and leave

R empty. A wire under R makes the button do nothing: R is the ring, which reaches jack pin 2, and nothing is connected there. Move that wire to L. Nothing can be damaged either way.

Check each test switch

1. Set the multimeter to continuity (the setting that beeps).
2. Hold one probe on the very end of the plug (the tip) and the other on the long metal part nearest the black handle (the sleeve). Nothing should beep.
3. Press the button: it should beep while pressed and stop when you let go.
4. If it beeps **only when not pressed**, one spade is on the NC tab: move it to NO. If it never beeps, check both spades are pushed fully on and both wires are clamped.

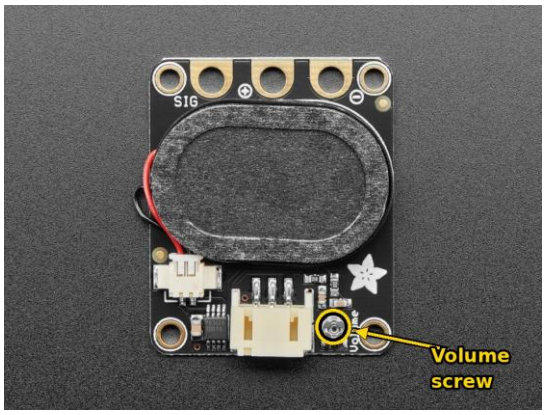
Label them, for example with a piece of tape: **DOT (plug into A)** and **DASH (plug into B)**. Either switch works in either jack; the label just keeps testing simple.

Holding or mounting the buttons

For testing, a caregiver can just hold the button in one hand and press it with the thumb. To mount it in a box or the enclosure, it needs a round hole **1.13 in (28.7 mm)** across, in a panel thin enough for the nut to grip behind it. Unscrew the nut, push the button through the hole from the front, and screw the nut back on from behind. The listing's drawing gives the sizes:

Sizes in inches, from the Amazon listing's drawing: mounting hole 1.13 in (28.7 mm); ring around the button 1.30 in (33 mm) across; about 2.4 to 2.6 in (61 to 66 mm) deep from the top of the button to the bottom of the switch.

Part 8: The speaker



The STEMMA Speaker. The cable plugs into the larger white socket at the bottom; the small one already holds the speaker's own wire. The volume screw (circled) is beside it, labeled Volume on the board. Photo: Adafruit; label added.



The speaker cable (#3893): white plug for the speaker, and white, red and black wires ending in pins. Photo: Adafruit.

1. Make sure the USB cable is unplugged.
2. Push the speaker cable's white plug into the socket on the speaker board. It only fits one way.
3. Push the cable's three pins into the Metro: **white** into **A0**, **red** into **3.3**, **black** into the **first GND** socket in the bottom row, next to VHI (jack A uses the second one).
4. The speaker's small **volume screw** sets the loudness. Leave it in the middle for now.

The speaker works from 3 to 5 volts, according to Adafruit, and runs from the 3.3 socket here.

Part 9: Check the wiring before plugging in

Go through this list with the USB cable still unplugged. Tick each line.

✓	**Check**
	Nothing is in **VHI** or **VIN**. Look twice at VIN: the GND next to it is used, VIN is not.
	The On/Off switch is at **ON**.
	Metro socket **5** has the wire from jack A's pin 5. Socket **6** has the wire from jack B's pin 5.
	Top-row **GND** has the wire from jack B's pin 1.
	Bottom row: **3.3** has the speaker's red wire, the first **GND** the speaker's black wire, the second **GND** the wire from jack A's pin 1.
	Metro **A0** has the speaker's white wire.
	Pins 2, 3 and 4 of both jack boards are empty, and no bare jack-board pin touches another pin or any metal.
	The sensor cable is pushed fully into both small sockets.
	Both test switches beep only while pressed (Part 7).

Every connection in this build was checked for voltage and current: none needs a resistor. The switches only connect a socket to ground when pressed, the sensor has its own cable, and the speaker has its own amplifier.

Part 10: Test everything

Put the test program on the Metro

1. Plug the USB cable back in. Wait for the **CIRCUITPY** drive to appear.
2. Open the file **code.py** on CIRCUITPY with a plain text editor (Notepad on Windows, TextEdit on Mac set to plain text). Don't use Word.
3. Delete everything in it, then copy the whole program below into it, exactly as printed. The spaces at the start of lines matter.
4. Save the file. The Metro restarts the program by itself every time code.py is saved.

The tested build skipped this test program and went straight to AeroMorse (Part 11), which worked. The program itself hasn't been run on hardware, so if it misbehaves, Part 11 is a reliable check.

```
# code.py: hardware test for the Metro AeroMorse (untested starter)
# Sockets match the AeroMorse program's defaults: dot D5, dash D6, speaker A0.
import time
import board
import digitalio
import pwmio
import adafruit_lps35hw  # driver for the LPS33HW and LPS35HW

# Dot and dash: jack A on D5, jack B on D6.
dot = digitalio.DigitalInOut(board.D5)
dot.switch_to_input(pull=digitalio.Pull.UP)
dash = digitalio.DigitalInOut(board.D6)
dash.switch_to_input(pull=digitalio.Pull.UP)

# Speaker on A0
spk = pwmio.PWMOut(board.A0, frequency=800, duty_cycle=0, variable_frequency=True)

def beep(freq, secs=0.08):
    spk.frequency = freq
    spk.duty_cycle = 2**15  # 50% square wave
    time.sleep(secs)
    spk.duty_cycle = 0

# Sensor. zero_pressure() makes readings relative to start-up:
# about 0 at rest, positive for a puff, negative for a sip.
lps = adafruit_lps35hw.LPS35HW(board.STEMMA_I2C())
lps.zero_pressure()

THRESH = 2   # hPa: AeroMorse's default sip / puff trigger
STRONG = 15  # hPa: AeroMorse's default strong sip / puff

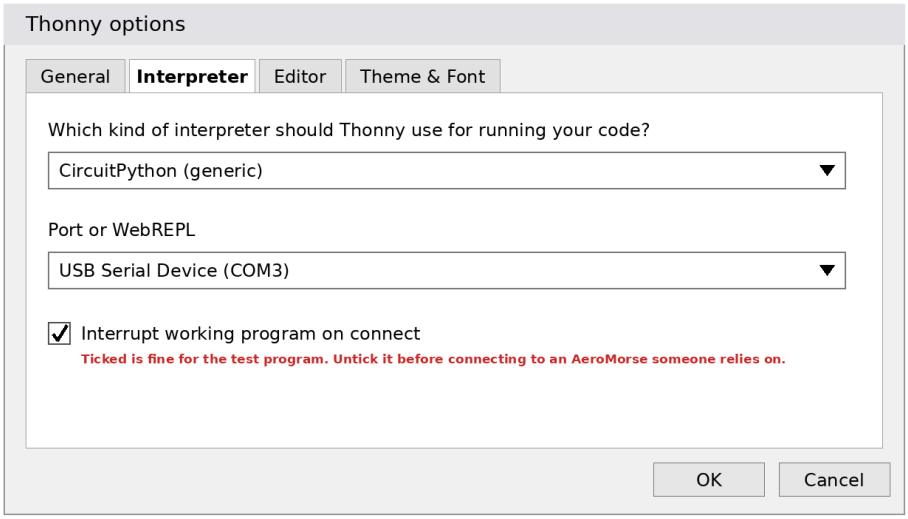
last = None
last_print = 0
```

```
while True:
    a = not dot.value   # True while pressed
    b = not dash.value
    if (a, b) != last:
        print("DOT", a, "DASH", b)
        if a:
            beep(1200)  # high beep for dot
        if b:
            beep(800)   # lower beep for dash
        last = (a, b)
    now = time.monotonic()
    if now - last_print >= 0.25:
        p = lps.pressure
        if p <= -STRONG:
            label = "STRONG SIP"
        elif p <= -THRESH:
            label = "sip"
        elif p >= STRONG:
            label = "STRONG PUFF"
        elif p >= THRESH:
            label = "puff"
        else:
            label = ""
        print("pressure %+.1f hPa %s" % (p, label))
        last_print = now
    time.sleep(0.02)
```

Open the serial console

The test program shows its results as text in the serial console. The AeroMorse Build Guide uses a free program called **Thonny** for this (its Appendix N).

1. Download Thonny from thonny.org and install it with the default options.
2. In Thonny, click **Run → Configure interpreter…**
3. In the top list choose **CircuitPython (generic)**. In **Port**, choose the Metro: on Windows a COM port such as COM3 (if unsure, open Device Manager → Ports and look for "USB Serial Device"); on a Mac something with usbmodem in the name.
4. Click **OK**. The **Shell** panel at the bottom shows `>>>`. That means Thonny has paused the program.
5. Click in the Shell panel and press **Ctrl+D**. The test program starts again and its lines appear in the Shell. Keep the tube still for the first second while the sensor sets its zero.



Simplified drawing of the settings named in AeroMorse Build Guide Appendix N, not a screenshot. Your port name will differ.

The Thonny settings from steps 2 and 3 (simplified drawing, not a screenshot).

What you should see



Example of what the test program prints. Your numbers will differ.

Example of the test program's output in Thonny's Shell panel.

1. **Pressure:** a line four times a second, about `pressure +0.0 hPa` at rest. Puff gently: the number goes up, and past +2 the line says `puff`. Sip gently: it goes negative and says `sip`. Blow or sip hard: past ±15 it says `STRONG PUFF` or `STRONG SIP`.
2. **Switch in jack A:** plug a test switch into jack A. Each press prints `DOT True DASH False` and gives a high beep. Letting go prints `DOT False DASH False`.
3. **Switch in jack B:** move it to jack B. Each press prints `DOT False DASH True` and gives a lower beep.
4. **Speaker too quiet or loud:** turn the volume screw gently with the small screwdriver.

What happens	What it means	What to do

What happens	What it means	What to do
No green ON light and no CIRCUITPY drive	The On/Off switch is at OFF, or the USB cable only charges	Slide the switch to ON; try a different USB-C cable
`ImportError: no module named 'adafruit_lps35hw'`	The sensor's library file isn't in CIRCUITPY's lib folder	Copy `adafruit_lps35hw.mpy` into lib (Part 3)
An error mentioning I2C or STEMMA	The sensor cable isn't pushed in fully	Unplug USB, reseat both ends, plug back in
`DOT True` repeats non-stop as soon as a switch is plugged in	The wire is on the jack's ring pin, which the switch's plug connects to ground	Unplug USB and move that wire to the pin your multimeter test found
Pressing a switch prints nothing	A jack wire isn't on the right pin or socket, or the plug isn't pushed fully into the jack	Push the plug in until it clicks; unplug USB and check against Part 9; check the test switch itself (Part 7)
No beeps	Speaker wire in the wrong socket, or volume turned right down	Check white = A0, red = 3.3, black = GND; turn the volume screw up
Pressure doesn't change when sipping or puffing	Tube not sealed on the sensor port, or not pushed fully into the joiner	Push the short tube further onto the port, and both tubes fully into the joiner (Part 5)

Part 11: Install AeroMorse

1. Go to github.com/jlubin2001/AeroMorse, click the green **Code** button and choose **Download ZIP**. Open the .zip and copy **boot.py**, **code.py**, **morse_map.py** and **config.py** onto CIRCUITPY. Replace the test program's code.py when asked.
2. Open **config.py** in the text editor and change these lines to read as below. Leave everything else as it is.
3. Save config.py, then **unplug the USB cable and plug it back in** so boot.py takes effect.
4. Open an empty Notepad window and click inside it. A short sip should type **e**, and a short puff **t**.
5. To try the switches, change `USE_SENSOR = True` to `USE_SENSOR = False` in config.py and save. Now a short press of the switch in jack A types **e**, and in jack B **t**. While `USE_SENSOR` is True the switches do nothing, which can look like a fault.

```
DEVICE_NAME = "AeroMorse Metro"   # any name you like
USE_SENSOR = True     # True = sip/puff, False = switches
USE_DISPLAY = False   # this board has no screen
USE_WIRELESS_DISPLAY = False  # True only with the wireless display (Part 13)
PC_DISPLAY = True     # show the display on the computer
```

Sip-and-puff or switches: AeroMorse uses one or the other. With `USE_SENSOR = True` it reads the tube and ignores the switches; with `False` it reads the switches and ignores the tube. Change the line and save; nothing needs rewiring. `DOT_PIN`, `DASH_PIN` and `AUDIO_PIN` already match the sockets used here.

Once AeroMorse is how someone uses the computer: connecting Thonny stops it. Before connecting Thonny to a working AeroMorse, open **Run → Configure interpreter…** and untick **Interrupt working program on connect**, or have another keyboard and mouse ready (Build Guide, Appendix N).

Part 12: How hard a sip or puff has to be

These lines in config.py set the sip-and-puff thresholds. The values below come from AeroMorse v1.33's config.py and from the config.py files of the author's own two devices (Green and Blue, v1.30). Start with the defaults and change one at a time.

Setting	Default	Green	Blue	What it does
`THRESH_SIP`	2	2	2	How far below resting pressure (hPa) a sip must go to count as a dot
`THRESH_PUFF`	2	2	1.5	How far above resting pressure a puff must go to count as a dash
`THRESH_SIP_STRONG` / `THRESH_PUFF_STRONG`	15	15	15	A strong sip or puff at the start of a code: strong sip jumps to the mouse group, strong puff to the keyboard group
`ACCEPT_DELAY`	0.3 s	0.2 s	0.2 s	How long to pause before a code is accepted
`REPEAT_SPLIT_PCT`	0 (off)	60	60	Splits two quick sips or puffs that run together
`BASELINE_DRIFT_S`	30	30	30	Lets the resting level follow slow weather changes while idle; 0 turns it off

- **If dots or dashes appear when nobody is sipping or puffing**, raise THRESH_SIP or THRESH_PUFF a little.
- **If gentle sips or puffs are missed**, lower them a little.
- **At start-up**, AeroMorse spends about a second measuring resting pressure. Keep the tube still while it starts.
- **To choose values for a new user**, copy `test_pressure.py` from the AeroMorse files to CIRCUITPY. In Thonny's Shell press Ctrl+C, type `import test_pressure` and press Enter. It shows the pressure for 30 seconds while they sip and puff normally, then suggests values.

Part 13 (optional): A wireless display

The Metro has no screen. The **ESP32-S3 Reverse TFT Feather** (#5691) is a second, smaller board with a colour screen built in. Set it where the user can see it, and it shows the same things AeroMorse would show on a screen: which group is active (keyboard, mouse and so on), the dots and dashes as they're entered, the last thing typed and the status. There are **no wires between the two boards**: the Metro sends this over a short-range radio link (called ESP-NOW) built into both boards. The AeroMorse Build Guide gives the range as about 30 m (100 ft) indoors.

There is nothing to assemble or solder. If a strip of metal pins comes in the bag with the board, put it aside: it isn't needed.

Work with one board plugged in at a time. Both boards show up as a drive called CIRCUITPY, so with both plugged in it's easy to copy a file to the wrong one. Unplug the Metro before you start.

Install CircuitPython on the display board

1. Plug the Feather into the computer with the USB-C **data** cable.
2. Find the small button labeled **Reset** on the Feather and press it twice quickly, like a double-click.
3. A new drive appears whose name ends in **BOOT**. If it doesn't, try the double-press again a little faster or slower.
4. In a web browser, go to circuitpython.org/board/adafruit_feather_esp32s3_reverse_tft and download the **.uf2** file for the latest stable version (10.3.1 when this guide was written).
5. Drag the .uf2 file onto the BOOT drive. The drive disappears, the board restarts, and after a few seconds a drive called **CIRCUITPY** appears. That means it worked.
6. If the copy fails, or no CIRCUITPY drive appears, the board's start-up program (called TinyUF2) may be too old: circuitpython.org says CircuitPython 10 needs version 0.33.0 or later on this board. The same page explains how to update it.

Copy the display files

1. On the Feather's CIRCUITPY drive, open the **lib** folder (create it and name it `lib` if there isn't one).
2. From the library bundle you downloaded in Part 3, copy **only** the folder `adafruit_display_text` into it. The display board needs nothing else from the bundle.
3. From the AeroMorse .zip you downloaded in Part 11, copy **boot.py** onto CIRCUITPY. It's the same file the Metro uses; it works out by itself which board it's on.
4. Copy **receiver.py** onto CIRCUITPY. Then delete the code.py that's already there and rename receiver.py to **code.py**. The board runs whatever is in code.py.
5. Copy **receiver_config.py** onto CIRCUITPY. It holds the display's own settings (see below).
6. **Don't copy morse_map.py** to this board. Its absence is how boot.py knows this board is a display and not a keyboard. With it, the computer would see a second keyboard.
7. Eject the CIRCUITPY drive and unplug the Feather.

When you're done, the Feather's drive should hold exactly this:

```
CIRCUITPY
├── boot.py              (same file as on the Metro)
```

```
├── code.py              (receiver.py, renamed)
├── receiver_config.py
└── lib
    └── adafruit_display_text   (folder)
```

Turn it on in the Metro

1. Plug the **Metro** back in (only the Metro).
2. Open **config.py** on its CIRCUITPY drive and change `USE_WIRELESS_DISPLAY = False` to `USE_WIRELESS_DISPLAY = True`.
3. Check the line `ESPNOW_CHANNEL = 1` just below it. It must be the **same number** as `ESPNOW_CHANNEL` in the Feather's receiver_config.py. Both are 1 to begin with, so leave them.
4. Save config.py. The Metro restarts by itself.

Check that it works

1. Power the Feather from a USB charger, a power bank or a spare USB port on the computer. Its screen shows **Waiting** until the Metro starts.
2. Within about a second of the Metro starting, the Feather's screen should show the AeroMorse display.
3. Give a sip or puff (or press a switch). The dot or dash should appear on the Feather's screen.

What you see	Likely cause	What to do
Waiting or No signal, never the display	`USE_WIRELESS_DISPLAY` is still False on the Metro, or the two `ESPNOW_CHANNEL` numbers differ	Check both settings, save, and unplug and replug the Metro
Screen stays blank	A file is missing or misnamed on the Feather	Plug the Feather into the computer, connect Thonny to it (as in Part 10) and read the Shell: an ImportError names the missing file
Computer sees an extra keyboard	morse_map.py was copied to the Feather	Delete it from the Feather's CIRCUITPY drive and unplug and replug the Feather

The display's own settings

These are in **receiver_config.py** on the Feather. Change a value and save; the display restarts with it. If the file is missing or a value is wrong, the display uses the default.

Setting	Default	What it does
`DISPLAY_BRIGHTNESS`	1.0	Screen brightness, from 0.1 (dim) to 1.0 (full)
`DISPLAY_ROTATION`	0	Which way up the picture is: 0 = USB socket on the left, 180 = USB socket on the right

Setting	Default	What it does
`SLEEP_AFTER_S`	300	Seconds with no signal before the screen goes dark. It wakes by itself when the Metro is back.
`ESPNOW_CHANNEL`	1	Must match the Metro's config.py

Not yet tested

The build in this guide works as a whole. These are the parts of the guide that weren't part of that test:

- The test program in Part 10. The tested build went straight to AeroMorse instead.
- The wireless display (Part 13) with the Metro. It is used with other AeroMorse boards; on the Metro it hasn't been tried. AeroMorse's config.py says the radio adds about 80–100 mA, which also comes from the Metro's 3.3 V supply alongside the speaker. If the Metro restarts during loud beeps with the wireless display on, turn the speaker's volume down.
- The speaker's current draw. Adafruit doesn't list it; from its amplifier chip's datasheet (PAM8302A) it should be about 240 mA at full volume on 3.3 V and 4–8 mA when silent, within the Metro's 3.3 V supply (about 400 mA usable). That's a calculation, not a measurement.
- Mounting the arcade buttons. The 1.13 in (28.7 mm) hole comes from the listing's drawing, which may show the original Happ part these buttons copy; measure yours before cutting the enclosure.

Sources

- Adafruit product pages: #5500, #4414, #4399, #3885, #3893, #5691
- Metro ESP32-S3 guide: pinouts, power management and the On/Off switch; Adafruit's labeled board photo
- Adafruit STEMMA Speaker guide and PAM8302A datasheet
- circuitpython.org: Metro ESP32-S3 (10.3.1, install steps)
- circuitpython.org: ESP32-S3 Reverse TFT Feather (10.3.1, TinyUF2 0.33.0 or later)
- Adafruit LPS33/LPS35 guide and LPS35HW library (zero_pressure)
- Adafruit sip-and-puff guide (tube attachment)
- Tubing on Amazon: B0H1WCK462 (uxcell silicone, 3 mm I.D. × 6 mm O.D., 1 m)
- Push-to-connect joiner on Amazon: B0H3T88BZD (ATMLCOER, 6 mm tube O.D., PU-6mm)
- Arcade buttons on Amazon: B07W2WLG2J (Fosiya, 30 mm; drawing with sizes)
- Mono plug adapters on Amazon: B0FJ1JVX9K (AKVIBG, 1/8" TS mono, + and − terminals)
- Jack boards on Amazon: B0FG24PBJJ (bent pins); pinout from the listing, read on 2026-11-04, and confirmed on the tested boards
- AeroMorse: Build Guide Appendices L (Metro) and N (Thonny), Step 9.2 (libraries), config.py v1.33, test_pressure.py, receiver.py and receiver_config.py v1.32, sections 5.3 and 9.4 (wireless display); Green and Blue config.py files